

- Description

This module manages a smart humidity control system for food storage, ensuring that:

1. The system continuously monitors the humidity level inside the storage room (expressed as a percentage).
2. If the humidity falls below a predefined minimum threshold, the humidifier is activated to increase humidity.
3. If the humidity exceeds a predefined maximum threshold, the dehumidifier is activated to reduce humidity.
4. If the humidity remains within the optimal range, no action is taken.

This system helps maintain optimal humidity levels for food preservation and prevents spoilage.

- Specification

module SmartFoodStorageHumidityControl

var currentHumidity, minHumidity, maxHumidity: real

var humidifierActive, dehumidifierActive: boolean

process updateHumidity(newHumidity: real) returns (currentHumidity: real, humidifierActive: boolean, dehumidifierActive: boolean)

functional scenario form:

currentHumidity < minHumidity && humidifierActive = true, dehumidifierActive = false, output "ALERT: Humidity too low (" + currentHumidity + "%). Humidifier activated.", Humidifier activated.

||

currentHumidity > maxHumidity && humidifierActive = false, dehumidifierActive = true, output "ALERT: Humidity too high (" + currentHumidity + "%). Dehumidifier activated.", Dehumidifier activated.

||

currentHumidity >= minHumidity && humidifierActive = false, dehumidifierActive = false, output "Humidity optimal (" + currentHumidity + "%). No action needed.", No control action required.

end process

end module

- Java Code

```
package code;
public class c112_SmartFoodStorage {
    private double minHumidity;
    private double maxHumidity;
    private double currentHumidity;
    private boolean humidifierActive;
    private boolean dehumidifierActive;
    public c112_SmartFoodStorage(double minHumidity, double maxHumidity
, double initialHumidity) {
```

```

        this.minHumidity = minHumidity;
        this.maxHumidity = maxHumidity;
        this.currentHumidity = initialHumidity;
        this.humidifierActive = false;
        this.dehumidifierActive = false;
    }

    public void updateHumidity(double newHumidity) {
        currentHumidity = newHumidity;
        adjustHumidityControl();
        System.out.printf("Current Humidity: %.2f%%, Humidifier: %s, Dehumidifier: %s\n",
            currentHumidity, humidifierActive ? "ON" : "OFF", dehumidifierActive ? "ON" : "OFF");
        if (humidifierActive) {
            System.out.println("Humidifier activated to increase humidity.");
        } else if (dehumidifierActive) {
            System.out.println("Dehumidifier activated to reduce humidity.");
        } else {
            System.out.println("Humidity is within the optimal range.");
        }
    }

    private void adjustHumidityControl() {
        if (currentHumidity < minHumidity) {
            humidifierActive = true;
            dehumidifierActive = false;
        } else if (currentHumidity > maxHumidity) {
            humidifierActive = false;
            dehumidifierActive = true;
        } else {
            humidifierActive = false;
            dehumidifierActive = false;
        }
    }

    public double getCurrentHumidity() {
        return currentHumidity;
    }

    public static void main(String[] args) {
        c112_SmartFoodStorage storage = new c112_SmartFoodStorage(50.0, 80.0, 65.0);
        // testcase-VT:
        // storage.updateHumidity(45.0);
    }

```

```
//      storage.updateHumidity(75.0);
//      storage.updateHumidity(85.0);

//      // testcase-FT:
//      storage.updateHumidity(-1.7976931344689044E308);
//      storage.updateHumidity(1.552019316280681E296);
//      storage.updateHumidity(1.3800241718403422E294);

//      // testcase-Z3:
//      storage.updateHumidity(45.0);
//      storage.updateHumidity(75.0);
//      storage.updateHumidity(85.0);

//      // testcase-UVT:
//      storage.updateHumidity(45.0);
//      storage.updateHumidity(75.0);
//      storage.updateHumidity(85.0);
//  }
}
```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.4 %	12	830	842
▼ src	1.4 %	12	830	842
▼ code	1.4 %	12	830	842
▼ c112_SmartFoodStorage.java	100.0 %	12	0	12
▼ c112_SmartFoodStorage	100.0 %	12	0	12
● updateHumidity(double)	100.0 %	8	0	8
■ adjustHumidityControl()	100.0 %	4	0	4
● c112_SmartFoodStorage(double)	100.0 %	0	0	0
● main(String[])	100.0 %	0	0	0

3. Vibration testing instruction coverage: 97.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	121	21,969	22,090
▼ src	0.5 %	121	21,969	22,090
▼ code	0.5 %	121	21,969	22,090
▼ c112_SmartFoodStorage.java	97.6 %	121	3	124
▼ c112_SmartFoodStorage	97.6 %	121	3	124
● updateHumidity(double)	100.0 %	53	0	53
■ adjustHumidityControl()	100.0 %	33	0	33
● c112_SmartFoodStorage(double)	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17

4. Fuzz testing branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	832	842
▼ src	1.2 %	10	832	842
▼ code	1.2 %	10	832	842
▼ c112_SmartFoodStorage.java	83.3 %	10	2	12
▼ c112_SmartFoodStorage	83.3 %	10	2	12
● updateHumidity(double)	87.5 %	7	1	8
■ adjustHumidityControl()	75.0 %	3	1	4
● c112_SmartFoodStorage(double)		0	0	0
● main(String[])		0	0	0

5. Fuzz testing instruction coverage: 90.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	112	21,978	22,090
▼ src	0.5 %	112	21,978	22,090
▼ code	0.5 %	112	21,978	22,090
▼ c112_SmartFoodStorage.java	90.3 %	112	12	124
▼ c112_SmartFoodStorage	90.3 %	112	12	124
● updateHumidity(double)	94.3 %	50	3	53
■ adjustHumidityControl()	81.8 %	27	6	33
● c112_SmartFoodStorage(double)	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.4 %	12	830	842
▼ src	1.4 %	12	830	842
▼ code	1.4 %	12	830	842
▼ c112_SmartFoodStorage.java	100.0 %	12	0	12
▼ c112_SmartFoodStorage	100.0 %	12	0	12
● updateHumidity(double)	100.0 %	8	0	8
■ adjustHumidityControl()	100.0 %	4	0	4
● c112_SmartFoodStorage(double)		0	0	0
● main(String[])		0	0	0

7. Z3 instruction coverage: 97.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	121	21,969	22,090
▼ src	0.5 %	121	21,969	22,090
▼ code	0.5 %	121	21,969	22,090
▼ c112_SmartFoodStorage.java	97.6 %	121	3	124
▼ c112_SmartFoodStorage	97.6 %	121	3	124
● updateHumidity(double)	100.0 %	53	0	53
■ adjustHumidityControl()	100.0 %	33	0	33
● c112_SmartFoodStorage(double)	100.0 %	18	0	18
● main(String[])	100.0 %	17	0	17

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.4 %	12	830	842
▼ src	1.4 %	12	830	842
▼ code	1.4 %	12	830	842
▼ c112_SmartFoodStorage.java	100.0 %	12	0	12
▼ c112_SmartFoodStorage	100.0 %	12	0	12
● updateHumidity(double)	100.0 %	8	0	8
■ adjustHumidityControl()	100.0 %	4	0	4
● c112_SmartFoodStorage(double)		0	0	0
● main(String[])		0	0	0

9. Unimproved V-method instruction coverage: 97.6%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  VibrationTestingTool		 0.5 %	121	21,969	22,090
▼  src		 0.5 %	121	21,969	22,090
▼  code		 0.5 %	121	21,969	22,090
▼  c112_SmartFoodStorage.java		 97.6 %	121	3	124
▼  c112_SmartFoodStorage		 97.6 %	121	3	124
 updateHumidity(double)		 100.0 %	53	0	53
 adjustHumidityControl()		 100.0 %	33	0	33
 c112_SmartFoodStorage(double)		 100.0 %	18	0	18
 main(String[])		 100.0 %	17	0	17